



MEDUSA PLUGIN · INSTALL GUIDE

Accept payments with **Kadima** on your Medusa store

```
npm install medusa-payment-kadima
```

 · card + ACH, no middle gateway

What you get

One plugin adds Kadima as a native payment method at checkout — cards and bank payments.



Cards

Visa/MC/Amex/Discover via Kadima Hosted Fields. Auth, capture, refund, saved cards.



ACH / eCheck

Bank debits with SEC codes (PPD/CCD/WEB/TEL). Settlement & returns via webhook.



PCI SAQ-A

Card data is entered in Kadima's secure iframe — it never touches your store or server.



No middle gateway

Process directly through Kadima — no NMI or extra gateway fees in between.



Saved cards & recurring

Store cards in Kadima's Customer Vault for one-click repeat & subscription billing.



Webhook reconciliation

Signed events keep order status in sync automatically (settled, refunded, returned).

Before you start

Two things: a Medusa v2 store, and a Kadima merchant account with API credentials.



A Medusa v2 store

Self-hosted Medusa v2 backend + a storefront (e.g. the Next.js starter). New to Medusa? Spin one up with:

```
npx create-medusa-app my-store
```

Requires Node 20+ and PostgreSQL.



Kadima credentials

A boarded Kadima merchant account. You'll create the token in Step 1; you'll also use:



API access token — created in Step 1 (scoped)



Terminal ID — your card MID/TID



DBA ID — for ACH + saved cards



Webhook secret — to verify signed events

Step 1 — Get your Kadima API credentials

Create a scoped API token in your Kadima dashboard — it authenticates every card & ACH call.

1

In the dashboard side menu (Help Center category), open Developers → Tokens.



Create the token

- 1 Click Add token.
- 2 Name it (e.g. "Medusa store") and pick your merchant.
- 3 Click Submit, then add the permissions (right).
- 4 Copy the token and store it securely — keep it secret, never commit it to source control.



Enable these permissions

- ✓ **Transactions / Payments** — card auth, capture, refund, void
- ✓ **ACH** — bank debits, status, void
- ✓ **Customer Vault** — saved cards & recurring
- ✓ **Webhooks / Events** — signed status events

Enable all of them so cards, ACH, saved cards and webhooks all work.

Step 2 — Install the plugin

One command in your Medusa backend project.

2

```
npm install medusa-payment-kadima
```

Published on npm — public package, MIT licensed.



What gets installed

- ✓ Two payment providers: kadima-card (cards) and kadima-ach (bank).
- ✓ Ready-to-use storefront components for the card + ACH forms.
- ✓ Hosted Fields, Customer Vault, and signed-webhook handling — built in.

Step 3 — Configure

Register both providers on the Payment module in medusa-config.ts.

```
// medusa-config.ts
modules: [{
  resolve: "@medusajs/medusa/payment",
  options: { providers: [
    { resolve: "medusa-payment-kadima/providers/kadima-card",
      id: "kadima-card", options: {
        apiToken: process.env.KADIMA_TOKEN,
        terminalId: Number(process.env.KADIMA_TERMINAL_ID),
        dbaId: Number(process.env.KADIMA_DBA_ID),
        webhookSecret: process.env.KADIMA_WEBHOOK_SECRET,
        captureMethod: "auth", sandbox: true } },
    { resolve: "medusa-payment-kadima/providers/kadima-ach",
      id: "kadima-ach", options: {
        apiToken: process.env.KADIMA_TOKEN,
        dbaId: Number(process.env.KADIMA_DBA_ID),
        secCode: "WEB", sandbox: true } },
  ] }
}]
```



Then

- ✓ Put your Kadima values in .env.
- ✓ Enable both providers in your region / sales channel.
- ✓ captureMethod: "auth" captures on fulfillment; "sale" captures now.
- ✓ Set sandbox explicitly — sandbox: true to test, false for live.



Don't derive sandbox from NODE_ENV — Railway/Render/Vercel set NODE_ENV=production, which sends sandbox creds to the live host (401). The plugin logs its resolved host + sandbox + token on startup — check that first.

Step 4 — Add the checkout components

Drop the included storefront components into your checkout's payment step.



KadimaHostedFields.tsx

Renders Kadima's secure card fields for the kadima-card provider. The customer enters the card in Kadima's iframe; on success your cart completes.



KadimaAchForm.tsx

Collects bank details for the kadima-ach provider, writes them to the payment session, and places the order (authorized; settles in 1–4 days).

Copy from the package's storefront/ folder:

```
// node_modules/medusa-payment-kadima/storefront/  
KadimaHostedFields.tsx  KadimaAchForm.tsx  README.md  (wiring guide)
```

Step 5 — Register the webhook

So settlements, refunds and ACH returns reconcile order status automatically.

5

In your Kadima dashboard, add a Webhook URL pointing to your Medusa app:

```
https://<your-store>/hooks/payment/kadima-card  
https://<your-store>/hooks/payment/kadima-ach
```

Event → what happens

 `transaction/create` → payment authorized / captured

 `transaction/refund` → refund recorded

 `ach/updateStatus` → `Settled` → ACH captured

 `ach/updateStatus` → `Returned` → order flagged / reversed

Step 6 — Test, then go live

Verify on sandbox, then flip to production.

Test (sandbox)

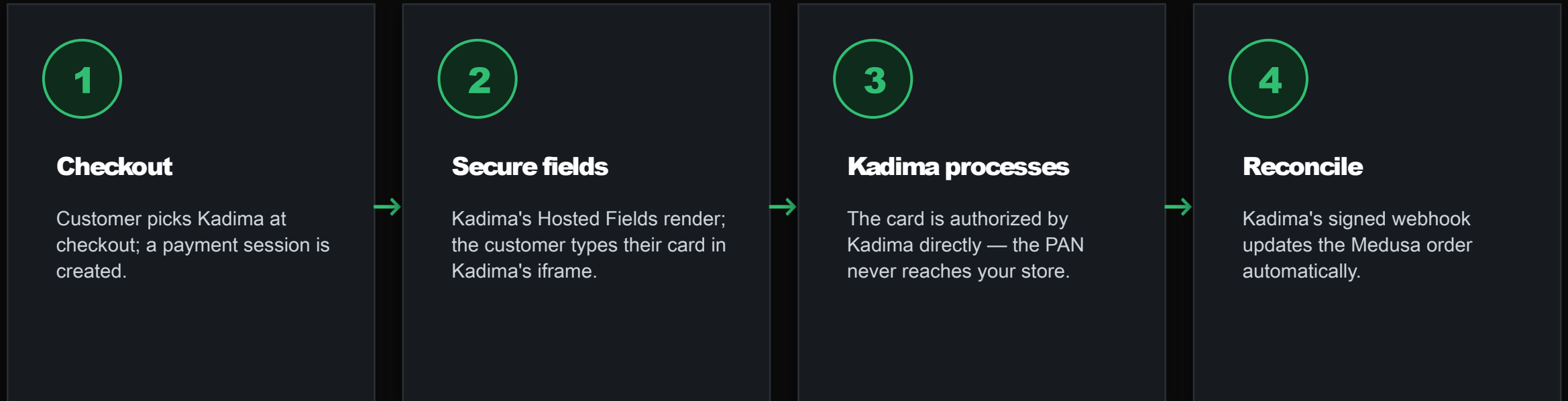
- ✓ Use Kadima sandbox credentials + a test card (4111 1111 1111 1111, 12/30, CVV 999).
- ✓ Run a full checkout: card and ACH.
- ✓ Confirm the order is paid and the webhook updates status.

Go live (production)

- 🚀 Set sandbox: false on both providers.
- 🚀 Swap in your live Kadima token + terminal/dba.
- 🚀 Point the webhook at your production URL.
- 🚀 Take a small real transaction to confirm.

How a card payment flows

Card data never touches your servers — you stay in PCI SAQ-A scope.



ACH works the same way, asynchronously: the order is placed as authorized, then settles or returns via webhook.



You're ready to accept payments

- ✓ **npm:** npmjs.com/package/medusa-payment-kadima
- ✓ **GitHub:** github.com/kadimapayments/medusa-payment-kadima
- ✓ **Storefront guide:** the storefront/ folder in the package
- ✓ **Support:** your Kadima rep · info@KadimaHQ.com

Card & ACH, straight through Kadima — no middle gateway.